RTK CRM security

English version of ../SECURITY.md. The document describes the implemented protection measures and compliance with Federal Law 152-FZ, FSTEC order No. 117 and Federal Law 149-FZ. The detailed results of the static analysis and the dependency review are in `security/SAST-SCA.md`.

Implemented measures

- JWT Bearer authentication with a limited token lifetime.
- RBAC: `user`, `manager`, `admin`; changes to reference data and the workflow are restricted by role.
- Passwords are stored only as bcrypt hashes.
- Database queries go through the SQLAlchemy ORM.
- Modifying requests are recorded in `action_logs`; sensitive fields (`password`, `token`, `secret`) are redacted before being written.
- CORS uses an explicit origin list, with no wildcard when credentials are involved.
- Exports and files are available to authorized users only.
- File uploads are limited by extension, MIME type and a 50 MB size.
- Sign-in attempt limiting: 5 failures per 60 seconds from one IP → `429`.
- Security headers on all responses: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, and in production `Strict-Transport-Security`. For the API additionally `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`; `/docs` and `/redoc` are excluded from the CSP because Swagger UI uses inline scripts.

Federal Law 152-FZ "On Personal Data"

Requirement	Implementation
Legal basis and consent	A cookie consent banner (`rtk_cookie_consent` in `localStorage`); mandatory and analytical categories are separated
Data minimisation	Stored are the working data of users (full name, e-mail, role) and of university contact persons (full name, phone, e-mail)
Database localisation	The target contour is Yandex Cloud (data in Russia). The public demo stand on external hosting is not intended for real personal data
Anonymisation	Exports contain only working identifiers and do not reveal credentials; the audit log redacts secrets
Retention periods	Determined by the operator; technically the audit log and `action_logs` support selective export and cleanup
Data protection officer	Appointed by the operating organisation

What is processed is the working data of users and university contact persons. In industrial operation the database and the backend must be placed in the Russian jurisdiction (Yandex Cloud), access to personal data must be restricted, retention periods must be defined and a data protection officer must be appointed. The public demo stand on external hosting should not be used for real personal data without a separate assessment and migration.

Cookie consent is stored in `localStorage` under the key `rtk_cookie_consent`; the analytical and marketing categories are disabled by default. The detailed migration plan to Yandex Cloud is in `infra/yandex-cloud/README.md`.

Data from the case holder

The case holder provided three files with export samples: `Вендоры.xlsx` (a catalogue of IT products with contacts), `Данные оплат.json` (B2C requests) and `Загрузка пользователей.xlsx` (an individual's questionnaire).

`Загрузка пользователей.xlsx` is a potentially sensitive file: besides the full name, phone and e-mail it contains columns for SNILS, passport series and number, registration address and diploma data. **In the copy provided those columns are empty** — only the full name, phone and e-mail are filled in, so the file holds no actual highly sensitive personal data.

Nevertheless the source exports do not enter the repository as they are: any tables with personal data are excluded by `.gitignore` rules (`*.xlsx`, `*.xls`, `Вендоры*`, `Данные оплат*`, `Загрузка пользователей*`). The reasons:

- the questionnaire is designed to collect SNILS and passport data, and any filled copy must not be committed;
- the prototype is not intended to store real passport data — highly sensitive fields are not implemented in the data model;
- test fixtures for the import are assembled in memory (`tests/test_excel_import.py`), so the source tables are not needed in the repository.

Instead of the source files the repository holds **anonymised samples** of the same formats (the "Data anonymisation" section below) — they are enough to demonstrate the import and contain no personal data.

Of those files only anonymised information reaches the demo data (`backend/seed.py`): the vendor catalogue and the products, plus B2C requests without contact data. Phones and e-mail addresses are replaced with synthetic values in the `example.ru` domain; the real names of contact persons are not stored.

Data anonymisation

The format samples needed to demonstrate the import were nevertheless added to the repository — in anonymised form, in the `docs/examples/` directory (mirrored in `frontend/public/docs/examples/`) and with Latin names so the files are served correctly by static hosting on Linux:

File	What is anonymised
`vendors.xlsx`	Vendor contacts — synthetic (`example.ru`)
`payments_b2c.json`	Names/phones of applicants — test data
`users_b2c.xlsx`	Personal-data columns replaced with placeholders

In `users_b2c.xlsx` every personal-data field is anonymised:

Field	Value
SNILS	`000-000-000 00`
Passport series and number	`0000, 000000`
Passport issued by	`ОТДЕЛОМ УФМС РОССИИ`
Passport issue date	`01.01.2020`
Department code	`000-000`
Registration address	`г. Москва, ул. Примерная, д. 1, кв. 1`
Registration postal code	`000000`

The full name, phone and e-mail are left as test data — fictitious values not relating to real individuals.

Rebuilding the anonymised copy from the original: `python scripts/anonymize_case_data.py <path to "Загрузка пользователей.xlsx">`. The original is stored outside the repository.

The `.gitignore` rules still exclude all exports with personal data (`*.xlsx`, `*.xls`, `Вендоры*`, `Данные оплат*`, `Загрузка пользователей*`); an exception was made narrowly for the anonymised samples in `docs/examples/`.

Verified: `git log --all --full-history -- "*Загрузка*"` — empty. The repository history contains no real personal data. Details — in `docs/examples/README.md`.

FSTEC order No. 117

Measure	Implementation
Identification and authentication	JWT with bcrypt password hashes; sign-in attempt limiting
Access control (RBAC)	Roles `user`/`manager`/`admin`, checked through FastAPI dependencies on every endpoint
Security event logging	`audit_middleware` → `action_logs`: user, action, entity, IP, time
Anti-virus protection	At the infrastructure level (documents are never executed on upload; files are served as attachments)
Integrity	Database constraints, foreign keys, Alembic migrations; uniqueness of workflow stages
Access to journals	Viewing the audit log and the CSV export are restricted to the administrator role
Channel encryption	HTTPS/TLS on the reverse proxy (Nginx); HSTS in production
Backup	A daily `pg_dump` in `infra/yandex-cloud/backup.sh`

Federal Law 149-FZ "On Information..."

It regulates the circulation of information in the system: the audit log provides traceability of changes, the role model provides separation of access to information resources, and data export provides a controlled format (CSV/JSON/XLSX). Integration with external LMS/CMS runs over HTTPS with explicit access keys.

Secrets and operation

`SECRET_KEY`, `DATABASE_URL` and other secrets are passed only through environment variables. In production the application rejects a default or too-short `SECRET_KEY`. `.env` files must never reach Git.

GigaChat

The card summary is the only call to an external AI service, so it has separate requirements.

- The authorization key (`GIGACHAT_CREDENTIALS`) is stored only in environment variables and in the `.env` on the server. It never reaches the repository, is never printed to logs and is never returned in API responses.
- The key is rotated in the GigaChat Developers console. Compromising the key gives no access to CRM data: the key is used only to request a summary.
- The TLS connection is verified against the Ministry of Digital Development CA root certificate. It is public, ships in `backend/certs/` and is baked into the image. Disabling verification (`GIGACHAT_VERIFY_SSL=false`) is not used in a working contour.
- Only names (university, product, stage), comment texts and service fields go into the prompt. Personal data (SNILS, passport, address, phone) is not stored in the card — it is dropped at the import stage, so it never reaches GigaChat.
- Access is checked before the prompt is assembled: a KAM cannot get a summary for someone else's interaction, and data from someone else's card never leaves for the external service.
- The service runs in Sber's closed contour and complies with 152-FZ: no personal data is transferred, and processing happens in Russia.

Details: `AI.md`.

Authentication: JWT and Keycloak

The specification names Keycloak as an authentication option. Both modes are implemented, switched by the `AUTH_MODE` variable:

`AUTH_MODE=jwt` (the default). The local contour for the prototype and the demonstration:

- `python-jose` signs Bearer tokens (HS256, the `SECRET_KEY` key);
- `passlib` uses bcrypt for passwords;
- RBAC is performed by FastAPI dependencies;
- the audit stores modifying operations without secret fields.

`AUTH_MODE=keycloak`. Corporate SSO for a closed contour:

- the token signature is verified against the realm's public keys (RS256, JWKS);
- keys are cached for 300 seconds, rotation is handled through an unknown `kid`;
- `iss`, `aud` and `exp` are verified;
- roles come from the token (`realm_access` and `resource_access`), so a role change in Keycloak takes effect immediately;
- a Keycloak user without a CRM account is created automatically with a role by priority admin > manager > kam; local password sign-in is closed for such an account.

Both modes return a local `User` record: it is needed for database relationships and RBAC, so the business logic does not depend on the authentication method.

If Keycloak is unavailable, the API answers `503`, not `401`: the token may be valid, and the client must not be logged out because of a network error.

Details: `KEYCLOAK.md`.

The `MOCK_MODE=true` variable pins the current JWT + database mode. There is no automatic switch to Keycloak when `MOCK_MODE=false` — the mode is set explicitly through `AUTH_MODE`, so that SSO is not enabled by accident when the demo-data flag changes.